F1 Race

DALI Multi-Agent System — Technical Documentation

Piccirilli Michael

March 3, 2026

Contents

1 Introduction **3**
1.1 DALI Overview . . . 3
1.2 Project Goals . . . 3

2 System Requirements **3**

3 Project Structure **4**

4 Agents **5**
4.1 Class Diagram . . . 5
4.2 DALI semantycs . . . 6
4.2.1 CarAgent: FerrariAgent, McLarenAgent, ... 6
4.2.2 PitWallAgent . . . 11
4.2.3 SemaphoreAgent . . . 16
4.2.4 SafetyCarAgent . . . 17
4.3 Custom FIPA Primitives (`communication.con`) . . . 19
4.3.1 `broadcast_to_list/3` — One-to-Many Inform . . . 19
4.3.2 `inform(telemetry/3)` — Diagnostic Telemetry . . . 19

5 Dynamic Agent System **20**
5.1 Code Generation Pipeline . . . 20
5.2 Adding or Removing a Car . . . 20

6 Race Flow **21**
6.1 Startup Sequence . . . 21
6.2 Lap Round-Robin . . . 21

7 Sequence Diagrams **22**
7.1 Startup Sequence . . . 22
7.2 Lap Round-Robin . . . 23
7.3 Safety Car Deployment . . . 24
7.4 Engine Failure (DNF) . . . 25
7.5 Push Lap (Internal Event) . . . 26

8 Timing System **27**

9 How to Run **27**
9.1 Native (WSL / Linux) — Not recommended 27
9.2 Web Dashboard — Recommended 27
9.2.1 Dashboard Controls 28
9.2.2 Circuit Tab 29

10 Docker Deployment **30**
10.1 Architecture 30
10.2 Quickstart 31
10.3 Environment Variables 31

11 DALI Source Patch System **31**
11.1 Motivation 31
11.2 Structure 32
11.3 Idempotency: the `[F1_PATCH]` Marker 32
11.4 Reverting the Patches 32

12 Shutdown **33**

Abstract

This document describes the design, architecture, and implementation of a Formula 1 race simulation built on top of the DALI Multi-Agent System framework. Car agents can be dynamically generated from a single agents.json configuration file. A dedicated agent coordinates the entire race flow and injects probabilistic track events (safety car deployment, heavy rain) at runtime. A real-time web dashboard provides live monitoring and manual control of the simulation. Finally, the project is entirely containerized using Docker, allowing for easy setup and execution across different environments.

1 Introduction

1.1 DALI Overview

DALI (Dynamic Agent Logic framework for Interaction) is a logic-based multi-agent system built on top of SICStus Prolog. Each agent is defined by a set of *external events* (reactions to incoming messages), *internal events* (proactive behaviours triggered by conditions), and standard Prolog clauses for knowledge representation and reasoning.

DALI agents run as independent SICStus processes that communicate via a shared message server (`LINDA`) running on a configurable TCP port.

1.2 Project Goals

The F1 Race simulation was developed with the following goals in mind:

- Demonstrate the *reactive* and *proactive* capabilities of DALI agents in a realistic, event-driven scenario.
- Show how a fully *dynamic* agent roster can be maintained through code generation, avoiding any hardcoded agent names in the DALI source.
- Provide a real-time monitoring interface that reflects agent activity without modifying the agent logic.
- Package the entire stack as a reproducible Docker Compose application.

2 System Requirements

Component	Requirement
SICStus Prolog	4.6.0 at `/usr/local/sicstus4.6.0` (Linux/WSL)
Shell	`bash`, `tmux`
Python	3.8+ (for `generate_agents.py` and dashboard)
Flask	Auto-installed in a local venv by `ui/run.sh`
Docker	Optional; required only for the Docker-based deployment

3 Project Structure

```
f1_race/
+-- agents.json           <- Configuration file for car agents
+-- generate_agents.py    <- Generates DALI files from agents.json
+-- startmas.sh           <- Launch script (Linux/WSL)
+-- startmas.bat          <- Launch script (Windows)
+-- docker-compose.yml    <- Docker Compose (mas + ui containers)
+-- .env.example          <- Template for .env file
+-- patch/                <- Runtime patches for DALI framework files
+-- docker/
|   +-- setup.sh          <- Auto-detects SICStus, writes .env
|   +-- mas/Dockerfile
|   \-- ui/Dockerfile
+-- ui/
|   +-- dashboard.py      <- Flask backend for the dashboard
|   +-- run.sh            <- Creates venv + launches dashboard
|   +-- requirements.txt
|   \-- static/
|       +-- index.html    <- Tab bar: Logs / Circuit
|       +-- app.js        <- Dashboard bootstrap, polling, tab switching
|       +-- circuit.js    <- Canvas circuit visualiser
|       \-- style.css     <- Layout, tab bar, circuit sidebar styles
+-- mas/
|   +-- instances/        <- Agent instance files
|   \-- types/            <- Agent type (behaviour) files
+-- conf/                 <- DALI communication configuration
+-- build/                <- Runtime binaries (auto-generated)
+-- work/                 <- Runtime working files (auto-generated)
\-- log/                  <- Agent logs (auto-generated)
```

4 Agents

4.1 Class Diagram

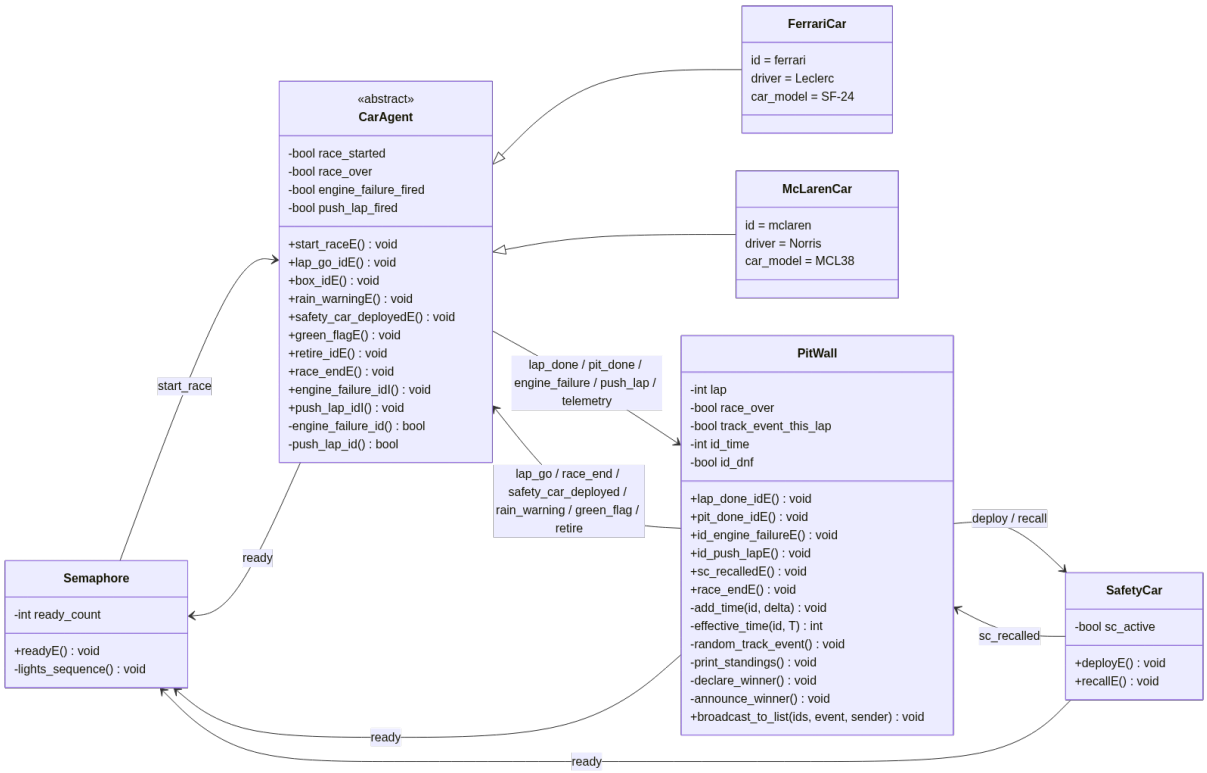

Figure 1: Class diagram of the F1 Race MAS.

The diagram models each DALI agent as a class (or equivalently a `type`) whose attributes represent its *persistent knowledge base* (Prolog facts asserted at runtime) and whose operations represent its *reactive* (external) and *proactive* (internal) event handlers.

CarAgent (one instance per car, dynamically generated) Holds the per-car racing state: `race_started` and `race_over` flags that gate the internal events; the `push_lap_{id}_fired` flag (reset every lap) that prevents the push-lap bonus from triggering more than once per lap; and the `engine_failure_{id}_fired` flag (never reset) that limits the DNF event to a single occurrence per race. Its external event handlers react to `start_race`, `lap_go`, `pit_done`, `race_end`, `safety_car_deployed`, `green_flag` and `retire`; its internal events `engine_failure_{id}I` and `push_lap_{id}I` fire autonomously when probabilistic conditions are met. `Id` are used to distinguish between the various instances of the *CarAgent*, such as **FerrariAgent** and **McLarenAgent**.

PitWallAgent Maintains the authoritative race state: the ordered list of car identifiers (used for round-robin lap dispatch), the `lap/1` counter, the `effective_time/2` facts that accumulate each car's total seconds, the `track_event_this_lap` flag (prevents more than one track event per lap), and the `race_over` flag. Its external handlers cover the full race protocol: `lap_done`, `pit_done`, `{id}_engine_failure`, `{id}_push_lap` and `sc_recalled`.

SemaphoreAgent Counts incoming `ready` messages via the `ready_count/1` fact. Once the count reaches $N_{\text{cars}} + 2$ (because the *PitwallAgent* and *SafetyCarAgent* are always present) it runs the lights sequence and sends `start_race` to first car. It has no internal events.

SafetyCarAgent Holds a single boolean fact `sc_active` toggled by the `deployE` and `recallE` external events. On recall it sends `sc_recalled` back to the pit wall to trigger the green-flag chain.

4.2 DALI semantycs

4.2.1 CarAgent: FerrariAgent, McLarenAgent, ...

The `FerrariAgent` is the canonical example of a car agent in the system. Its source file `mas/types/ferrariCar.txt` is generated by `generate_agents.py` from the `agents.json` entry for `ferrari`. All other car agents follow the same structure with agent-specific identifiers and driver names substituted.

Initialisation Directive

```
:- write('[Ferrari] SF-24 ready on the grid.'),
   send_m(semaphore, send_message(ready, ferrari)).
```

This *directive* (introduced by `:-`) executes at agent load time, before any event handler is registered. It immediately sends a `ready` message to the `semaphore` agent, contributing to the $N_{\text{cars}} + 2$ count that triggers the lights sequence. The four `dynamic` declarations that follow initialise the agent's mutable knowledge base:

```
:- dynamic race_started/0.
:- dynamic race_over/0.
:- dynamic engine_failure_ferrari_fired/0.
:- dynamic push_lap_ferrari_fired/0.
```

All four facts are absent at startup; they are asserted at runtime by event handlers and serve as boolean flags.

External Events

External events (`nameE:> Body.`) are *reactive*: the DALI runtime fires them whenever a message whose name matches `name` arrives in the agent's mailbox.

`start_raceE` — Lights Out

```
start_raceE:>
    assert(race_started),
    retractall(push_lap_ferrari_fired),
    write('[Ferrari] LAP 1 -- LIGHTS OUT! Leclerc launches off
       the line!'),
    messageA(pitwall, send_message(lap_done_ferrari, ferrari)).
```

Sent by the `semaphore` after the lights-out pause.

1. Asserts `race_started`, enabling the internal-event conditions.
2. Retracts, i.e. removes all clauses that can be unified with the fact, any stale `push_lap_ferrari_fired` flag (defensive reset for the first lap).
3. Immediately reports the first lap as done to the pit wall via `messageA`.

`lap_go_ferrariE` — Next Lap Dispatch

```
lap_go_ferrariE:>
    if(\+ race_started, assert(race_started), true),
    retractall(push_lap_ferrari_fired),
    write('[Ferrari] On the power! Leclerc attacking every
       sector.'),
    messageA(pitwall, send_message(lap_done_ferrari, ferrari)).
```

Sent by the pit wall at the start of every subsequent lap.

1. The `if` guard handles the edge case where `start_race` arrived out of order or was missed: it re-asserts `race_started` defensively.
2. Retracts `push_lap_ferrari_fired`, resetting the push-lap window so the proactive event can fire again this lap.
3. Reports the lap immediately as done.

`box_ferrariE` — Pit Stop

```
box_ferrariE:>
   write('[Ferrari] BOX BOX BOX! Ferrari dives into the pits.'),
   messageA(pitwall, send_message(pit_done_ferrari, ferrari)).
```

Sent by the pit wall when it decides to call Ferrari in for a stop. Immediately acknowledges the stop with `pit_done_ferrari` so the pit wall can add the +25 s penalty and proceed with lap dispatch.

`rain_warningE` and `safety_car_deployedE` — Track Notifications

```
rain_warningE:>
    if(race_over, true,
        (write('[Ferrari] RAIN WARNING. Leclerc switching to
           intermediate tyres.'), nl)).

safety_car_deployedE:>
    if(race_over, true,
        (write('[Ferrari] Safety car deployed. Leclerc
           conserving tyres.'), nl)).
```

Both handlers are *cosmetic*: they never modify the knowledge base and are silently ignored (branch `true`) if the race has already ended. The `race_over` guard prevents spurious output from late-arriving messages after `race_endE` has fired.

`green_flagE` — Restart After Safety Car

```
green_flagE:>
    if(race_over, true,
        (write('[Ferrari] GREEN FLAG! Leclerc pushing flat out!'
           ), nl)).
```

Sent by the pit wall once the safety car has been recalled (`sc_recalled` chain). Purely cosmetic; guarded by `race_over`.

`retire_ferrariE` — DNF Notification

```
retire_ferrariE:>
    write('[Ferrari] *** LECLERC PARKS THE CAR. Ferrari is out
       of the race. ***').
```

Sent by the pit wall after receiving an `engine_failure` message from this agent. No state change is needed here because `race_over` is only asserted by `race_endE`, which the pit wall broadcasts to all agents immediately after `declare_winner`.

`race_endE` — Race Termination

```
race_endE:>
    assert(race_over).
```

Broadcast by the pit wall to every agent after `declare_winner`. Asserting `race_over` causes the conditions of both internal events (`engine_failure_ferrari` and `push_lap_ferrari`) to fail permanently, stopping all proactive behaviour.

Internal Events

Internal events (`nameI:> Body.`) are *proactive*: the DALI runtime evaluates the associated condition clause `name :- Cond.` at each agent cycle. If the condition succeeds, the body fires.

`engine_failure_ferrariI` — Spontaneous DNF

```
engine_failure_ferrari :-
    race_started,
    \+ race_over,
    \+ engine_failure_ferrari_fired,
    random(0, 1000, R), R < 2.

engine_failure_ferrariI:>
    assert(engine_failure_ferrari_fired),
    write('[Ferrari] *** ENGINE FAILURE -- Power loss! Leclerc
       slowing! ***'),
    send_m(pitwall, send_message(ferrari_engine_failure, ferrari
       )),
    send_m(pitwall, inform(telemetry(ferrari, engine_failure,
       critical), ferrari)).
```

The condition requires four conjuncts to hold simultaneously:

- `race_started` — the race is live.
- `\+ race_over` — the race has not yet ended.
- `\+ engine_failure_ferrari_fired` — the event has not already fired (once-only guard).
- `random(0, 1000, R), R < 2` — a 0.2 % probability roll succeeds.

On firing, `engine_failure_ferrari_fired` is asserted first to prevent re-entry, then `send_m` notifies the pit wall. Note the use of `send_m` (safe inside non-top-level code) rather than `messageA` (top-level only). A second `send_m` delivers a FIPA `inform` with a `telemetry/3` payload (`telemetry(ferrari, engine_failure, critical)`) at priority 100, logged by the pit wall as a diagnostic trace. The pit wall reacts via `ferrari_engine_failureE`, sets the effective time to 9999 s, and eventually broadcasts `race_end`.

`push_lap_ferrariI` — Fastest-Lap Bonus

```
push_lap_ferrari :-
    race_started,
    \+ race_over,
    \+ push_lap_ferrari_fired,
    random(0, 100, R), R < 10.

push_lap_ferrariI:>
    assert(push_lap_ferrari_fired),
    write('[Ferrari] PUSH LAP! Leclerc going flat out -- setting fastest sectors!'),
    send_m(pitwall, send_message(ferrari_push_lap, ferrari)),
    send_m(pitwall, inform(telemetry(ferrari, push_lap, active), ferrari)).
```

Structurally identical to the engine-failure event but with a 10 % probability. The `push_lap_ferrari_fired` flag is retracted at the start of every lap by `lap_go_ferrariE` (and `start_raceE`), so the event fires *at most once per lap* rather than at most once per race. A FIPA `inform` with payload `telemetry(ferrari, push_lap, active)` is also sent at priority 100 for diagnostic tracing at the pit wall. The pit wall handler subtracts 3 s from Ferrari's accumulated time upon receiving `ferrari_push_lap`.

Knowledge Base and Event Summary

Name	Description
Knowledge base	
`race_started`	Set on lights-out; gates all lap and event handlers
`race_over`	Set on `race_endE`; silences all guarded handlers
`engine_failure_ferrari_fired`	One-shot flag; prevents engine-failure from firing twice
`push_lap_ferrari_fired`	Per-lap flag; retracted by `lap_go_ferrariE` each lap
External events	

Name	Description
`start_raceE`	Lights-out from semaphore; asserts `race_started`, sends `lap_done_ferrari`
`lap_go_ferrariE`	Pit wall starts a new lap; resets push-lap flag, sends `lap_done_ferrari`
`box_ferrariE`	Pit-stop request; sends `pit_done_ferrari` (+25 s penalty)
`rain_warningE`	Heavy-rain notification (cosmetic, guarded by `race_over`)
`safety_car_deployedE`	Safety-car deployment notification (cosmetic, guarded by `race_over`)
`green_flagE`	Safety-car recalled; car acknowledges green flag (guarded by `race_over`)
`retire_ferrariE`	DNF confirmation from pit wall; car parks
`race_endE`	Race-over broadcast; asserts `race_over`
Internal events	
`engine_failure_ferrariI`	$\approx 0.2\,\%$/cycle; fires once per race; notifies pit wall of DNF; sends `inform(telemetry(ferrari, engine_failure, critical))`
`push_lap_ferrariI`	$\approx 10\,\%$/cycle; fires once per lap; deducts 3 s via pit wall; sends `inform(telemetry(ferrari, push_lap, active))`

4.2.2 PitWallAgent

The `PitWallAgent` is the race coordinator of the simulation. Its source file `mas/types/pitWallType.txt` is generated by `generate_agents.py` and embeds the full car roster: it knows every car by name and drives the round-robin lap dispatch[1]. Unlike car agents, it has no internal events: in fact, all its behaviour is reactive.

Initialisation Directives

```
:- write('[PitWall] Online.'),
   send_m(semaphore, send_message(ready, pitwall)).
:- dynamic ferrari_time/1.
:- dynamic ferrari_dnf/0.
:- dynamic mclaren_time/1.
:- dynamic mclaren_dnf/0.
:- dynamic lap/1.
:- dynamic race_over/0.
:- dynamic track_event_this_lap/0.
:- assert(ferrari_time(0)).
:- assert(mclaren_time(0)).
:- assert(lap(0)).
```

At load time the agent registers with the semaphore and seeds the knowledge base. For each car a `{id}_time/1` accumulator is initialised to zero and a `{id}_dnf/0` flag is declared (absent until a DNF occurs). `lap/1` holds the current completed-lap count, starting at zero.

Knowledge-Base Helpers

The agent defines three sets of pure Prolog predicates that are called from within event handlers.

`add_time/2` and `effective_time/2`

```
add_time(ferrari, D) :-
    retract(ferrari_time(S)),
    S1 is S + D,
    assert(ferrari_time(S1)).

effective_time(ferrari, T) :- ferrari_dnf, !, T = 9999.
effective_time(ferrari, T) :- ferrari_time(T).
```

`add_time/2` performs an atomic read-modify-write on the accumulator: it retracts the old value, computes the new one, and re-asserts it. Passing a negative delta (e.g. -3) subtracts time. `effective_time/2` first checks the DNF flag with a cut, avoiding the execution of the other predicate; if the car has retired it returns a sentinel value of 9999 (a simbolic infinite time registered) regardless of the actual accumulator.

[1] This concept is explained better in Section 7.2

`random_track_event/0`

```
random_track_event :-
    if(track_event_this_lap, true,
        (random(0, 10, R),
         if(R < 2,
             (assert(track_event_this_lap),
              write('[Race Director] SAFETY CAR deployed. +10s to all.'), nl,
              add_time(ferrari, 10), add_time(mclaren, 10),
              send_m(safety_car, send_message(deploy, pitwall)),
              broadcast_to_list([ferrari, mclaren], safety_car_deployed, pitwall)),
         if(R < 4,
             (assert(track_event_this_lap),
              write('[Race Director] HEAVY RAIN. +5s to all.'), nl,
              add_time(ferrari, 5), add_time(mclaren, 5),
              broadcast_to_list([ferrari, mclaren], rain_warning, pitwall)),
             true)))).
```

The outer `if` is an idempotency guard: if `track_event_this_lap` is already asserted the predicate returns immediately, ensuring at most one track event per lap. Otherwise a uniform random integer $R \in [0, 10)$ is drawn:

- $R < 2$ (20 %): safety car deployed, +10 s to all cars.
- $2 \leq R < 4$ (20 %): heavy rain, +5 s to all cars.
- $R \geq 4$ (60 %): no event.

`declare_winner/0` and `announce_winner/0`

```
declare_winner :-
    if(race_over, true,
        (assert(race_over),
         write('[PitWall] === CHEQUERED FLAG ==='), nl,
         announce_winner,
         send_m(ferrari, send_message(race_end, pitwall)),
         send_m(mclaren, send_message(race_end, pitwall)))).

announce_winner :-
    findall(T-Id, effective_time(Id, T), Pairs0),
    keysort(Pairs0, Pairs),
    write('[PitWall] === FINAL RESULTS ==='), nl,
    print_podium(Pairs, 1).
```

`declare_winner/0` is guarded by `race_over` to prevent double invocation (it can be triggered by either the last `lap_done` or an engine-failure event). It asserts `race_over`, prints the podium via `announce_winner/0`, and broadcasts `race_end` to all cars using `broadcast_to_list/3` (defined in `communication.con`) instead of individual `send_m`

calls, so the number of cars is never hardcoded here. `announce_winner/0` uses `findall` to collect all `effective_time` pairs, sorts them by key (time) with `keysort`, and delegates formatted output to `print_podium/2`.

External Events

`lap_done_ferrariE` — Ferrari Lap Completion

```
lap_done_ferrariE:>
    if(race_over, true,
        (random(60, 91, T),
         add_time(ferrari, T),
         write('[PitWall] Ferrari lap: '), write(T), write('s'), nl,
         random_track_event,
         messageA(mclaren, send_message(lap_go_mclaren, pitwall))).
```

Triggered when Ferrari reports a completed lap.

1. Draws a random lap time $T \in [60, 90]$ s and adds it to Ferrari's accumulator.
2. Calls `random_track_event` to potentially inject a safety car or rain event.
3. Dispatches the next lap token to McLaren via `messageA`.

`lap_done_mclarenE` — McLaren Lap Completion and Lap Counter

```
lap_done_mclarenE:>
    if(race_over, true,
        (random(60, 91, T),
         add_time(mclaren, T),
         write('[PitWall] McLaren lap: '), write(T), write('s'), nl,
         (track_event_this_lap
             -> (retract(track_event_this_lap),
                 send_m(safety_car, send_message(recall, pitwall)))
             ; true),
         retract(lap(N)), N1 is N + 1, assert(lap(N1)),
         write('[PitWall] Lap '), write(N1), write(' / 5'), nl,
         print_standings,
         if(N1 =:= 5, declare_winner,
             (random_track_event,
              messageA(ferrari, send_message(lap_go_ferrari, pitwall))))))).
```

McLaren is last in the round-robin, so this handler carries the end-of-lap bookkeeping:

1. Adds McLaren's lap time.
2. If a track event occurred this lap, retracts the flag and sends `recall` to the safety car (triggering the `sc_recalled` chain).

3. Increments the lap counter and prints the current standings.
4. If the total lap count equals 5, calls `declare_winner`; otherwise rolls another `random_track_event` and dispatches `lap_go_ferrari` to start the next lap.

`pit_done_ferrariE` and `pit_done_mclarenE` — Pit Stops

```
pit_done_ferrariE:>
    if(race_over, true,
        (add_time(ferrari, 25),
         write('[PitWall] Ferrari pit stop +25s.'), nl,
         messageA(mclaren, send_message(lap_go_mclaren, pitwall)
            ))).

pit_done_mclarenE:>
    if(race_over, true,
        (add_time(mclaren, 25),
         write('[PitWall] McLaren pit stop +25s.'), nl,
         retract(lap(N)), N1 is N + 1, assert(lap(N1)),
         if(N1 =:= 5, declare_winner,
             messageA(ferrari, send_message(lap_go_ferrari,
                pitwall))))).
```

Each handler adds the +25 s pit-stop penalty and then resumes the round-robin exactly as its `lap_done` counterpart would. Note that `pit_done_mclarenE` also increments the lap counter because a pit stop replaces the normal lap-done report for the last car in the round.

`sc_recalledE` — Safety Car Recalled

```
sc_recalledE:>
    write('[Race Director] GREEN FLAG! Track is clear.'), nl,
    broadcast_to_list([ferrari, mclaren], green_flag, pitwall).
```

Sent by the `SafetyCarAgent` after receiving `recall`. The pit wall announces the green flag and forwards it to every car so their `green_flagE` handlers can fire.

`ferrari_engine_failureE` and `mclaren_engine_failureE` — DNF

```
ferrari_engine_failureE:>
    assert(ferrari_dnf),
    write('[PitWall] Ferrari DNF. Leclerc is out.'), nl,
    send_m(ferrari, send_message(retire_ferrari, pitwall)),
    declare_winner.

mclaren_engine_failureE:>
    assert(mclaren_dnf),
    write('[PitWall] McLaren DNF. Norris is out.'), nl,
    send_m(mclaren, send_message(retire_mclaren, pitwall)),
    declare_winner.
```

Triggered by a car's `engine_failure_{id}I` internal event. The handler asserts the DNF

flag (causing `effective_time` to return 9999 for that car), sends the cosmetic `retire_{id}` message back to the car, and calls `declare_winner` immediately, ending the race for all participants.[2]

`ferrari_push_lapE` and `mclaren_push_lapE` — Fastest Lap Bonus

```
ferrari_push_lapE:>
    if(race_over, true,
        (add_time(ferrari, -3),
         write('[PitWall] Ferrari fastest lap! -3s'), nl)).

mclaren_push_lapE:>
    if(race_over, true,
        (add_time(mclaren, -3),
         write('[PitWall] McLaren fastest lap! -3s'), nl)).
```

Triggered by a car's `push_lap_{id}I` internal event. Calls `add_time` with $\Delta = -3$ to subtract three seconds from the car's accumulator. Guarded by `race_over` to ignore late-arriving push-lap messages after the race has ended.

Knowledge Base and Event Summary

Name	Description
Knowledge base	
`ferrari_time/1`	Accumulated race time for Ferrari (seconds)
`mclaren_time/1`	Accumulated race time for McLaren (seconds)
`ferrari_dnf/0`	Asserted on engine failure; makes `effective_time` return 9999
`mclaren_dnf/0`	Asserted on engine failure; makes `effective_time` return 9999
`lap/1`	Current completed-lap counter; starts at 0
`race_over/0`	Asserted by `declare_winner`; gates all handlers
`track_event_this_lap/0`	One-shot flag; prevents multiple track events per lap
External events	
`lap_done_ferrariE`	Adds lap time, rolls track event, dispatches to McLaren
`lap_done_mclarenE`	Adds lap time, recalls SC if needed, increments lap, checks race end
`pit_done_ferrariE`	Adds +25 s, dispatches to McLaren
`pit_done_mclarenE`	Adds +25 s, increments lap, checks race end
`sc_recalledE`	Safety car recalled; broadcasts `green_flag` to all cars via `broadcast_to`
`ferrari_engine_failureE`	Asserts DNF, retires car, triggers `declare_winner`
`mclaren_engine_failureE`	Asserts DNF, retires car, triggers `declare_winner`
`ferrari_push_lapE`	Subtracts 3 s from Ferrari's accumulator
`mclaren_push_lapE`	Subtracts 3 s from McLaren's accumulator

[2]This is a known limitation of the current implementation: any DNF unconditionally terminates the entire race, regardless of how many cars are still running. A more realistic design would call `declare_winner` only when all but one car have retired, and would remove the DNF car from the round-robin dispatch chain. Both changes would require the `PitWallAgent` to maintain a dynamic list of active cars rather than a hardcoded sequence of named message sends.

4.2.3 SemaphoreAgent

The `SemaphoreAgent` is the race starter. Its source file `mas/types/semaphoreType.txt` is generated by `generate_agents.py` and encodes the exact number of agents that must check in before the lights sequence can begin. It has no internal events and only one external event handler.

Initialisation Directives

```
:- write('Semaphore: Waiting for all F1 agents to be ready...'), nl.
:- dynamic ready_count/1.
:- if(clause(ready_count(_), _), true, assert(ready_count(0))).
```

The agent declares a single mutable counter `ready_count/1` and initialises it to 0 only if it is not already present. The `if(clause(...),true,assert(...))` idiom is a safe guard against double-initialisation that could arise if the directive were executed more than once.

Knowledge-Base Helper

`lights_sequence/0`

```
lights_sequence :-
    write('Semaphore: ========================================'), nl,
    write('Semaphore: F1 RACE START SEQUENCE INITIATED'), nl,
    write('Semaphore: ========================================'), nl,
    sleep(1), write('Semaphore:  (O) ( ) ( ) ( ) ( )  -- light 1'), nl,
    sleep(1), write('Semaphore:  (O) (O) ( ) ( ) ( )  -- light 2'), nl,
    sleep(1), write('Semaphore:  (O) (O) (O) ( ) ( )  -- light 3'), nl,
    sleep(1), write('Semaphore:  (O) (O) (O) (O) ( )  -- light 4'), nl,
    sleep(1), write('Semaphore:  (O) (O) (O) (O) (O)  -- light 5'), nl,
    sleep(2), write('Semaphore:  *** LIGHTS OUT! ***'), nl,
    write('Semaphore:  ( ) ( ) ( ) ( ) ( )  -- GO GO GO!'), nl,
    write('Semaphore: ========================================'), nl,
    send_m(ferrari, send_message(start_race, semaphore)).
```

The predicate implements the canonical F1 start procedure: five red lights come on one per second ($5 \times 1\,\mathrm{s}$), hold for 2 s, then go out. All timing is achieved with blocking `sleep/1` calls, which is acceptable here because the semaphore has no other concurrent duties once the lights are running. On completion it sends `start_race` to the first car (`ferrari`), which triggers the round-robin lap dispatch.

External Events

`readyE` — Agent Registration

```
readyE:>
    retract(ready_count(N)),
    retractall(ready_count(_)),
    N1 is N + 1,
    assert(ready_count(N1)),
    write('Semaphore: '), write(N1), write('/'), write(4),
    write(' agents ready.'), nl,
    if(N1 =:= 4, lights_sequence, true).
```

Sent by every agent at load time (cars, pit wall, safety car). Each arrival:

1. `retract(ready_count(N))` removes one clause and binds `N` to the current value, necessary because `retractall` discards clauses without binding any variable and therefore cannot be used alone to read the counter. The subsequent `retractall(ready_count(_))` then eliminates any duplicate clauses that may have accumulated if two `ready` messages were processed concurrently before either had time to run its own `retract`.
2. Increments the counter and re-asserts it.
3. Prints the current progress (e.g. `2/4 agents ready`).
4. When the count reaches the threshold (here hardcoded to 4, generated from `agents.json` as $N_{\text{cars}} + 2$), calls `lights_sequence/0` to start the race; otherwise waits for the next `ready` message.

Knowledge Base and Event Summary

Name	Description
Knowledge base	
`ready_count/1`	Running count of `ready` signals received; initialised to 0
External events	
`readyE`	Increments the counter; fires `lights_sequence` when threshold is reached

4.2.4 SafetyCarAgent

The `SafetyCarAgent` is the simplest agent in the system. Its source file `mas/types/safetyCarType.txt` is generated by `generate_agents.py`. It manages a single boolean flag (`sc_active`) and acts as a stateful relay in the safety-car message chain between the pit wall and the car agents.

Initialisation Directives

```
:- write('Safety Car ready. Standing by.'), nl,
   send_m(semaphore, send_message(ready, safety_car)).
:- dynamic sc_active/0.
```

At load time the agent registers with the semaphore and declares the `sc_active/0` flag, which is absent until a `deploy` message arrives.

External Events

`deployE` — Safety Car Deployment

```
deployE:>
    if(sc_active, true,
        (assert(sc_active),
         write('SAFETY CAR: DEPLOYED! Yellow flags! All cars
            reduce speed!'), nl)).
```

Sent by the pit wall when `random_track_event` rolls a safety-car result. The `if(sc_active, true, ...)` guard makes the handler idempotent: if the safety car is already deployed a second `deploy` message is silently ignored. Otherwise `sc_active` is asserted and the deployment is logged.

`recallE` — Safety Car Recalled

```
recallE:>
    if(sc_active,
        (retract(sc_active),
         write('SAFETY CAR: Returning to pits. Notifying race
            control.'), nl,
         send_m(pitwall, send_message(sc_recalled, safety_car)))
            ,
        true).
```

Sent by the pit wall at the end of the lap during which the safety car was deployed. The guard here is inverted: the recall is only processed if the safety car is actually active. On recall:

1. `sc_active` is retracted, clearing the deployment state.
2. `sc_recalled` is sent back to the pit wall, triggering `sc_recalledE` there, which in turn broadcasts `green_flag` to all cars.

This completes the three-step safety-car chain: `random_track_event` (PitWall) → `recallE` (SafetyCar) → `sc_recalledE` (PitWall) → `green_flagE` (CarAgents).

Knowledge Base and Event Summary

Name	Description
Knowledge base — dynamic facts	
`sc_active/0`	Present while the safety car is deployed; absent otherwise
External events	
`deployE`	Asserts `sc_active`; idempotent if already deployed
`recallE`	Retracts `sc_active`, sends `sc_recalled` to pit wall

4.3 Custom FIPA Primitives (communication.con)

The file conf/communication.con extends the standard set of DALI told/tell rules with two F1-specific FIPA primitives. Both are active for every agent in the MAS because communication.con is loaded globally at agent startup.

4.3.1 broadcast_to_list/3 — One-to-Many Inform

```
broadcast_to_list([], _, _).
broadcast_to_list([H|T], Content, Sender) :-
    send_m(H, send_message(Content, Sender)),
    broadcast_to_list(T, Content, Sender).

told(Ag, inform(broadcast(Targets, Content, Sender)), 70) :-
    write('[Broadcast] from '), write(Sender), write(' to '),
       write(Targets), nl,
    write('[Broadcast] content: '), write(Content), nl,
    broadcast_to_list(Targets, Content, Sender),
    !.

tell(To, _, inform(broadcast(_, _, _))) :- true.
```

A recursive helper that sends the same send_message to every agent in Targets. The companion told/3 rule at priority 70 (equal to the standard inform priority) handles the case where an agent *receives* an inform(broadcast(...)) message, it logs the broadcast and delegates to the helper, cutting after the first successful match.

In the generated pitWallType.txt, every multi-car notification is expressed with this primitive rather than with repeated individual send_m calls:

```
broadcast_to_list([ferrari, mclaren], safety_car_deployed,
   pitwall).
broadcast_to_list([ferrari, mclaren], rain_warning,
   pitwall).
broadcast_to_list([ferrari, mclaren], green_flag,
   pitwall).
broadcast_to_list([ferrari, mclaren], race_end,
   pitwall).
```

Because the list is built from agents.json at generation time, adding or removing a car from the roster requires no manual edits to the broadcast calls.

4.3.2 inform(telemetry/3) — Diagnostic Telemetry

```
told(Ag, inform(telemetry(Car, Data, Level)), 100) :-
    write('[Telemetry] '), write(Car),
    write(' | '), write(Data),
    write(' | '), write(Level), nl,
    !.

tell(To, _, inform(telemetry(_, _, _))) :- true.
```

A high-priority (100) `told` rule that intercepts any `inform` whose content is a `telemetry/3` term. The three arguments are:

- `Car`: the originating car identifier (*e.g.* `ferrari`).
- `Data`: the event type (`engine_failure` or `push_lap`).
- `Level`: severity or mode (`critical` or `active`).

Car agents send this message alongside the standard `send_message` in their `engine_failure_{id}I` and `push_lap_{id}I` handlers. The receiving agent (pit wall) logs the payload via `write` and stores it in its `past_event` database via the standard `call_inform` infrastructure. No reactive logic is currently triggered on receipt; the record can be queried retrospectively.

5 Dynamic Agent System

5.1 Code Generation Pipeline

The `generate_agents.py` script is the central point for generating all DALI files: it reads `agents.json` and automatically generates configuration files for each agent (cars, pit wall, semaphore, and safety car).

```
agents.json
    +-> generate_agents.py
          +-- mas/instances/{id}.txt        (one per car)
          +-- mas/types/{id}Car.txt         (one per car)
          +-- mas/types/pitWallType.txt     (round-robin
             over all cars)
          +-- mas/types/semaphoreType.txt   (waits N_cars +
             2 ready)
          \-- mas/types/safetyCarType.txt
```

`generate_agents.py` is invoked automatically at each MAS startup by `startmas.sh`. It has a smart way to regenerate files:

Situation	Behaviour
`agents.json` unchanged	Skips regeneration — no files written
New car added	Full regeneration
Car removed	Stale `{id}.txt` / `{id}Car.txt` deleted, then full regeneration
`--force` flag	Always regenerates unconditionally

5.2 Adding or Removing a Car

Edit `agents.json` and add a new entry to the `cars` array:

```
{
  "total_laps": 5,
  "cars": [
```

```
    { "id": "ferrari",
      "team": "Ferrari",
      "car_model": "SF-24",
      "driver": "Leclerc",
      "label": "Ferrari SF-24",
      "color": "#180505",
      "border": "#cc2200"
    },
    { "id": "myclubcar",
      "team": "My Club",
      "car_model": "X1",
      "driver": "Rossi",
      "label": "Club X1",
      "color": "#001020",
      "border": "#00aaff"
    }
  ]
}
```

Then run `bash startmas.sh`. All DALI files and the dashboard update automatically.

6 Race Flow

6.1 Startup Sequence

1. Every agent (all cars, pit wall, safety car) sends a `ready` message to the semaphore.
2. The semaphore waits until it has collected exactly $N_{\text{cars}} + 2$ ready signals.
3. The semaphore runs the F1 lights sequence (5 lights on, 1 s each; 2 s pause; lights out) and sends `start_race` to `Car[0]`.

6.2 Lap Round-Robin

Once the race starts, laps proceed in a strict round-robin over the ordered car list:

1. `Car[i]` receives `lap_go_{id}` from the pit wall.
2. `Car[i]` asserts its lap status and sends `lap_done_{id}` to the pit wall.
3. The pit wall adds a random lap time, optionally fires a track event, and sends `lap_go_{id+1}` to the next car.
4. After the last car of a lap completes:
   - Lap counter increments.
   - Standings are printed.
   - If laps remain: `random_track_event` is rolled, then `lap_go` is sent to `Car[0]`.
   - If race complete: `declare_winner` is called and `race_end` is broadcast to all cars via `broadcast_to_list/3`.

At any point a car's *internal events* (`engine_failureI` or `push_lapI`) can fire autonomously and notify the pit wall.

7 Sequence Diagrams

The following diagrams illustrate the main message-passing flows between agents. Each arrow represents a `send_m` or `messageA` call in the DALI source.

7.1 Startup Sequence

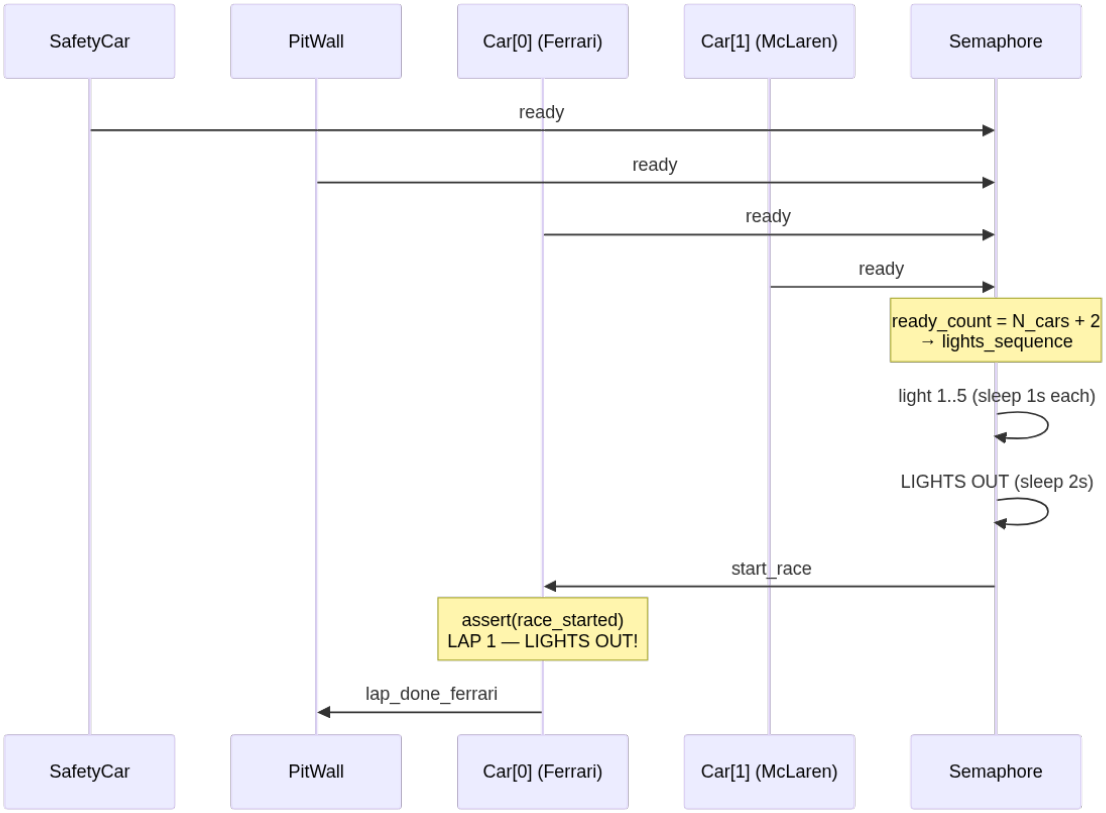

Figure 2: Startup sequence: all agents register with the semaphore, which runs the lights sequence and fires `start_race` to the first car.

7.2 Lap Round-Robin

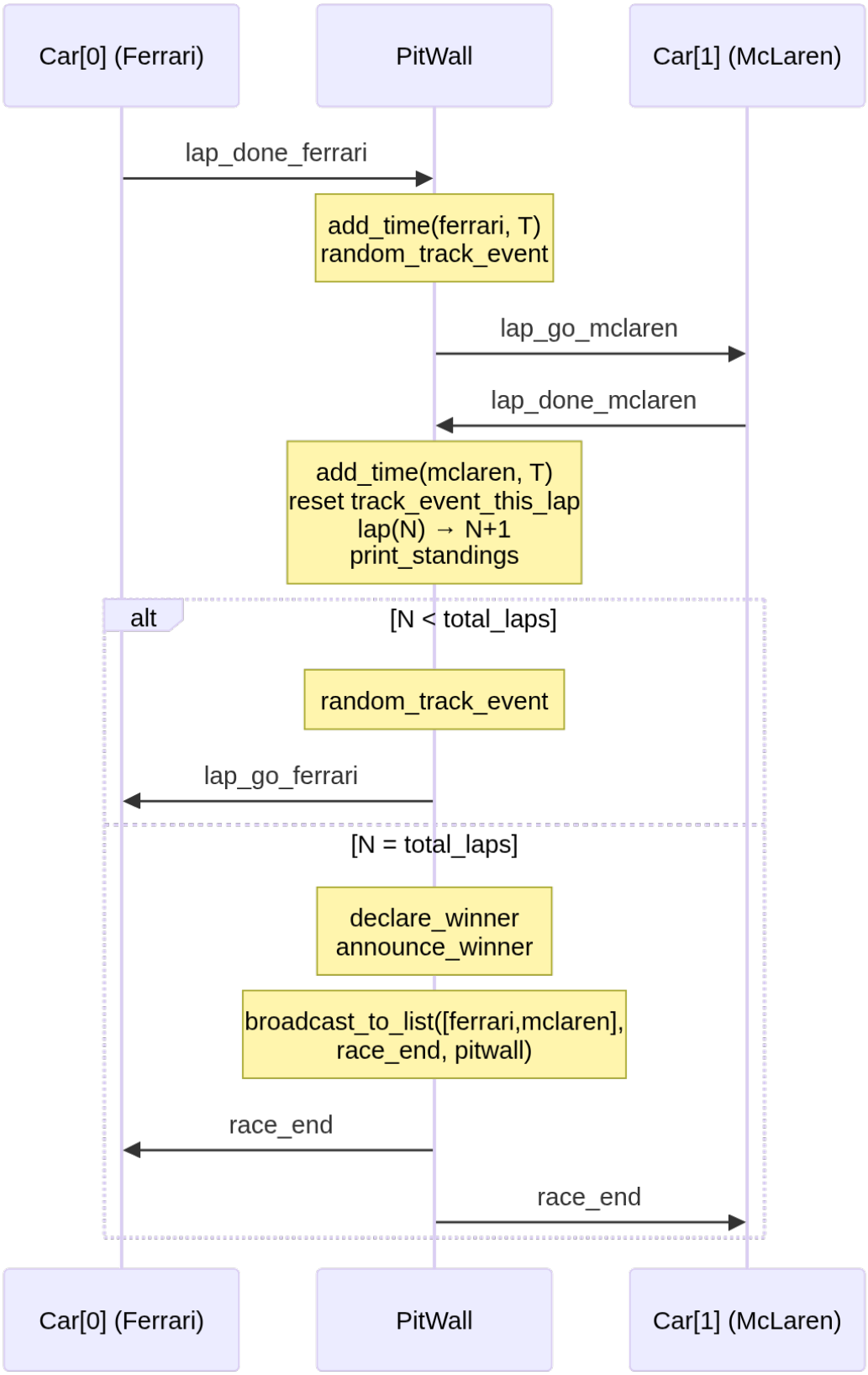

Figure 3: Lap round-robin: the pit wall dispatches `lap_go` tokens in order, accumulates lap times, and increments the lap counter after the last car.

7.3 Safety Car Deployment

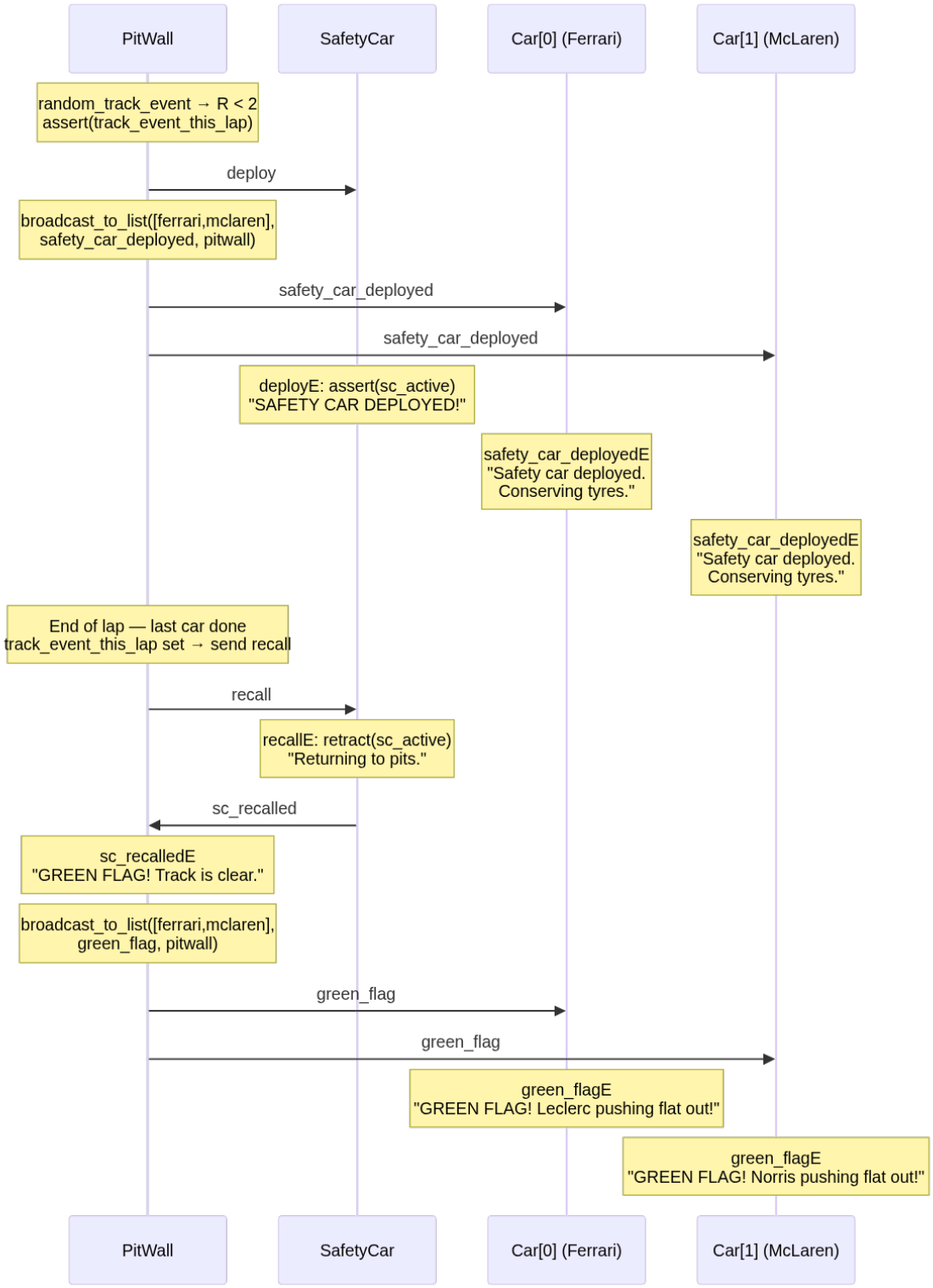

Figure 4: Safety car chain: pit wall deploys the safety car and notifies all cars via `broadcast_to_list`; at the end of the lap the safety car is recalled and the green flag is broadcast via `broadcast_to_list`.

7.4 Engine Failure (DNF)

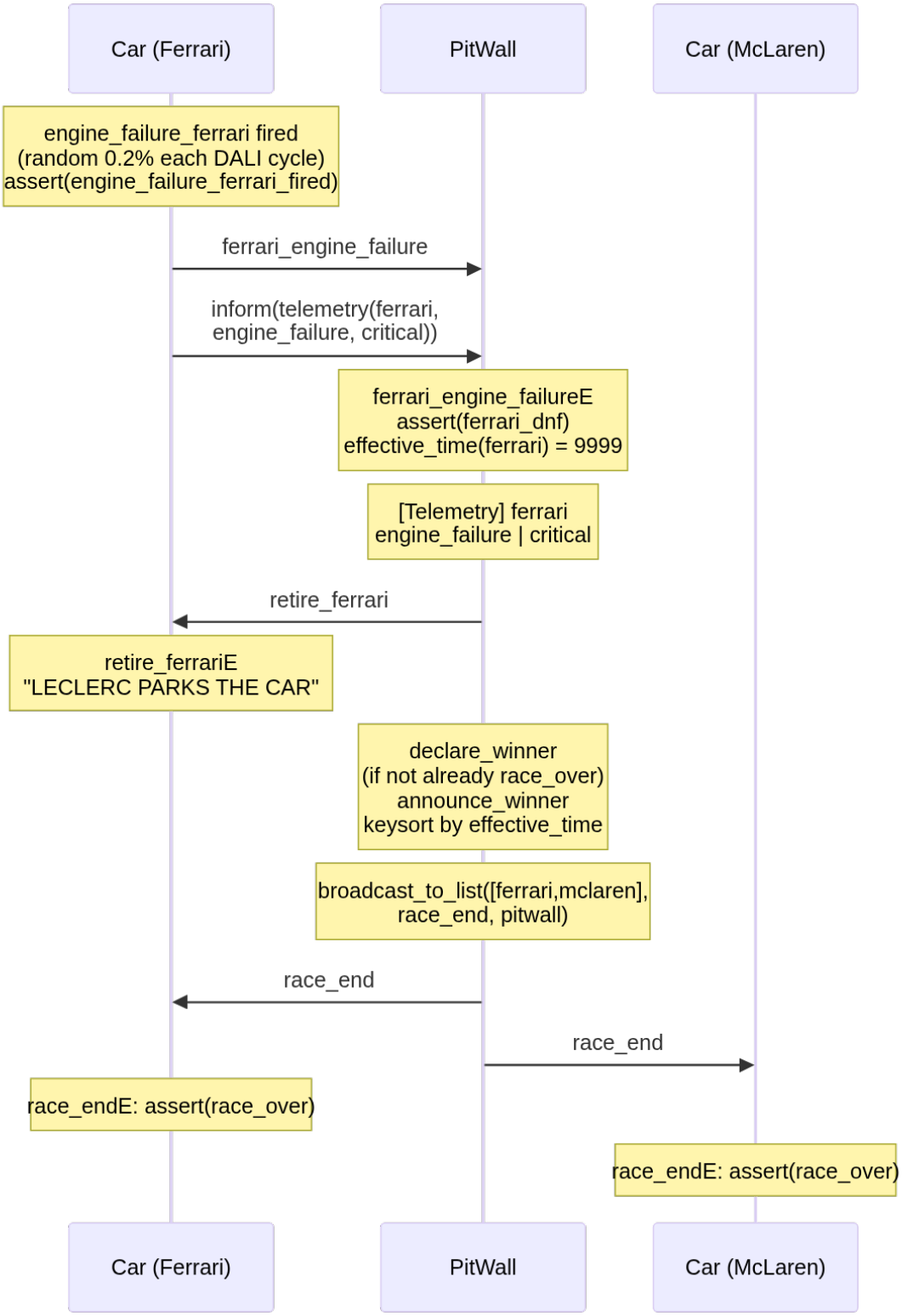

Figure 5: Engine failure: a car's internal event notifies the pit wall via `send_message` and a FIPA `inform(telemetry)` at priority 100; the pit wall sets the DNF time, retires the car, and immediately calls `declare_winner`, broadcasting `race_end` via `broadcast_to_list`.

7.5 Push Lap (Internal Event)

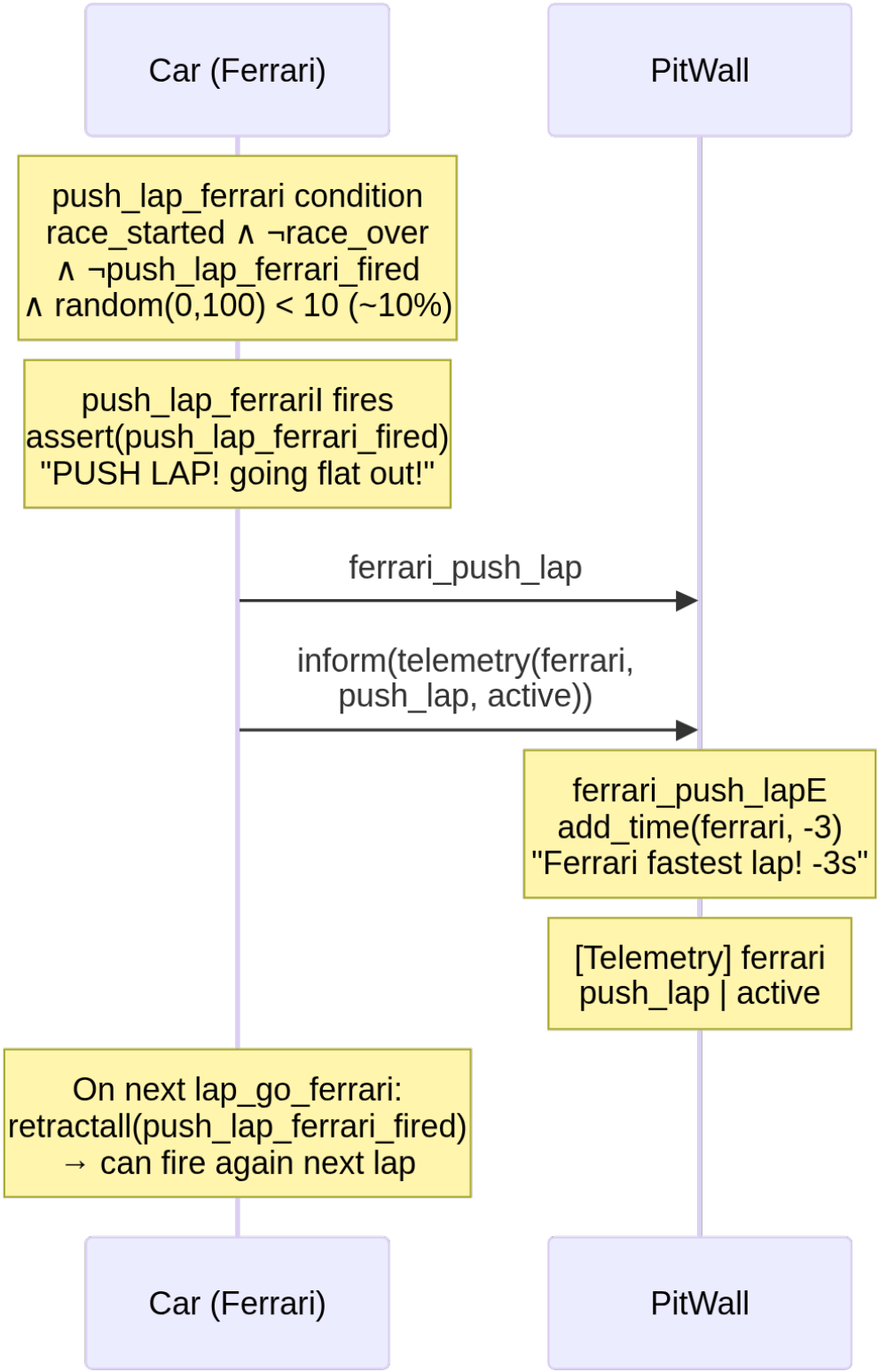

Figure 6: Push lap: a car's internal probabilistic event (≈ 10 %/cycle) fires at most once per lap, notifies the pit wall via `send_message` and a FIPA `inform(telemetry)` at priority 100 for diagnostic tracing; the pit wall subtracts 3 s from the car's accumulator.

8 Timing System

The winner is the car with the lowest total accumulated time. Time is measured in seconds.

Event	Time change
Each lap	+ `random`(60, 90) seconds
Pit stop	+ 25 s
Safety car (20 % per lap)	+ 10 s to *all* cars
Heavy rain (20 % per lap)	+ 5 s to *all* cars
Push lap (internal event, ≈10 %)	− 3 s
Engine failure / DNF (≈0.2 %)	`time` = 9999 s (race ends)

9 How to Run

9.1 Native (WSL / Linux) — Not recommended

The configurations can be a little messy because requires the SICStus interpreter and the DALI logic, but here are presente a step-by-step guide:

1. Clone the DALI repo from `https://github.com/AAAI-DISIM-UnivAQ/DALI` and navigate to the `DALI/Examples` folder.
2. Clone the GitHub repo project from `https://github.com/Mik1810/f1_race` under `Examples` folder.

Then, execute the following commands in the terminal:

```
cd DALI/Examples/f1_race
bash startmas.sh
```

`startmas.sh` performs the following steps:

1. Kills any stale SICStus / tmux processes on port 3010-3019.
2. Runs `generate_agents.py` to sync DALI files with `agents.json`.
3. Launches all agents inside a single `tmux` session named `f1_race`.

9.2 Web Dashboard — Recommended

If you want to use the web dashboard, which provides real-time logs and an animated circuit view, run the following commands in terminal:

```
cd f1_race
bash ui/run.sh
```

`run.sh` creates a local Python venv (`ui/.venv`) on the first run and installs Flask automatically. Subsequent launches or skip `pip install` via a stamp file (`.venv/.installed_stamp`) and start it.

Open `http://localhost:5000` in a browser. The dashboard shows all agent panes side-by-side with real-time auto-scrolling. If `agents.json` is modified while the dashboard is open, it auto-detects the change within 5 seconds and rebuilds the grid without a page reload.

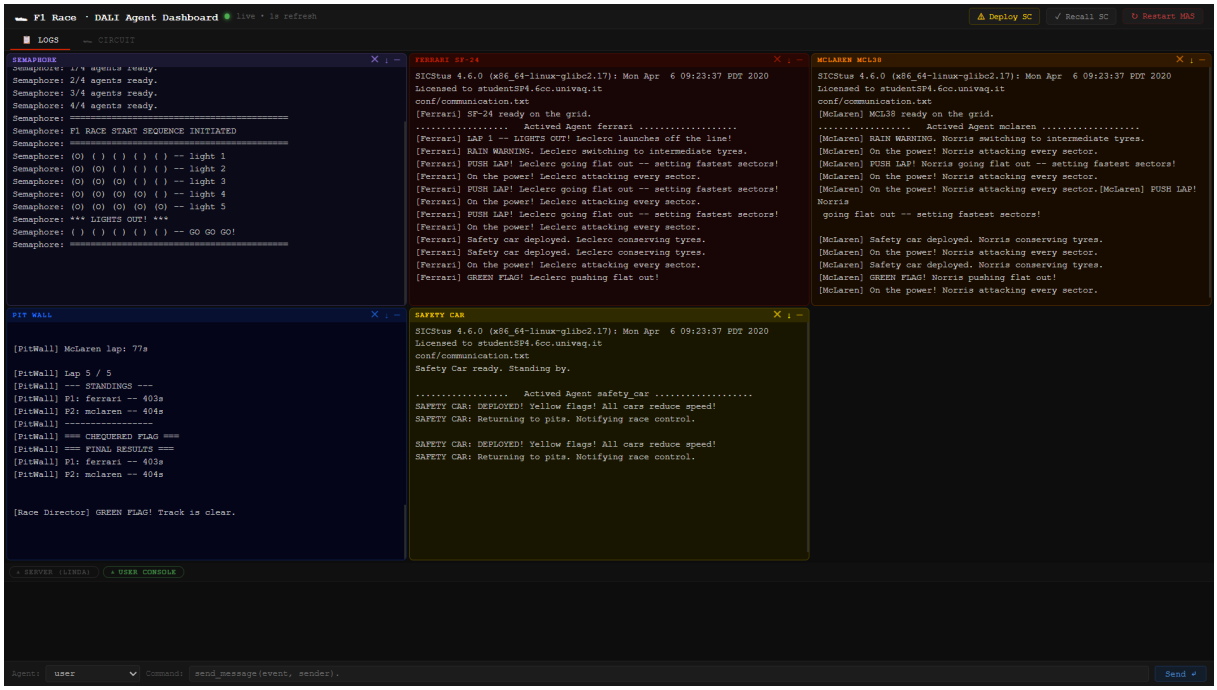

Figure 7: Web dashboard — Logs tab: agent panes with real-time output, control buttons and command bar.

9.2.1 Dashboard Controls

UI element	Function
Restart MAS	Kills SICStus + tmux session, reruns `startmas.sh`
Deploy SC	Sends deploy message to safety car immediately
Recall SC	Recalls the safety car
Agent / Command bar	Sends an arbitrary Prolog command to any agent pane
Pin button (↓)	Toggles auto-scroll for that pane
Clear button (×)	Clears pane output
Minimise button (−)	Minimises pane to tray

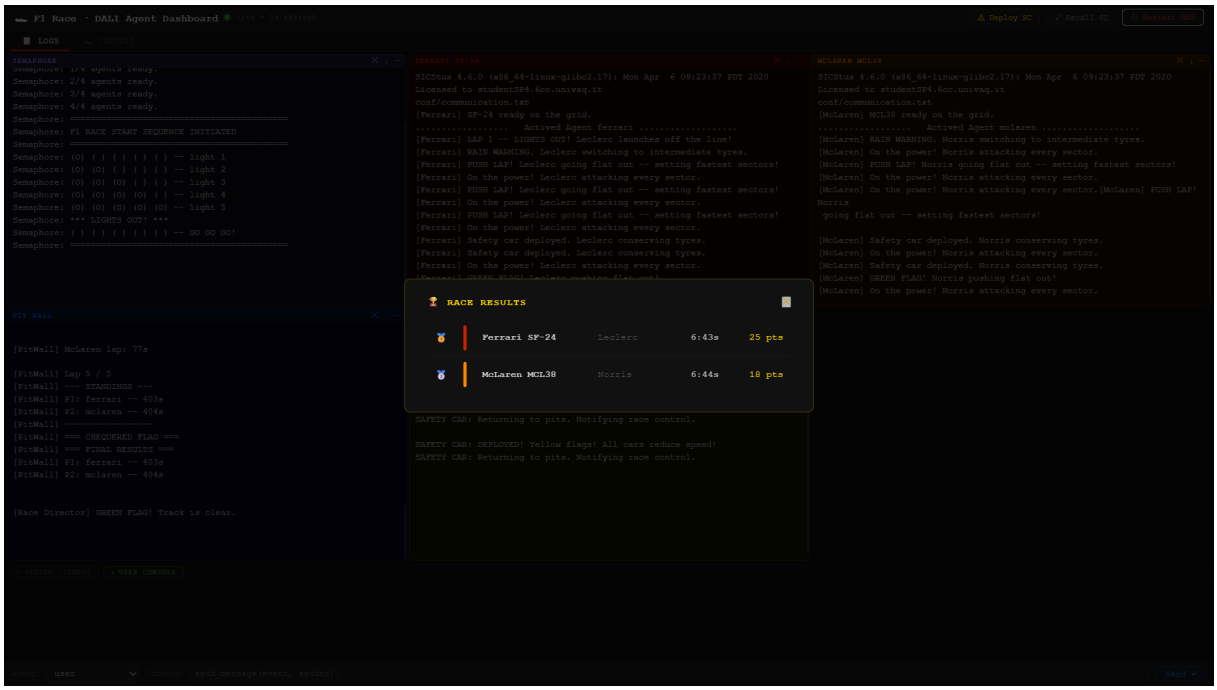

Figure 8: Final podium modal displayed at race end, showing driver positions and total accumulated times.

9.2.2 Circuit Tab

The dashboard exposes two tabs selectable via the tab bar at the top of the page:

- **Logs**: the default view showing all agent panes side-by-side with real-time output.
- **Circuit**: an animated canvas visualisation of the race circuit.

Clicking a tab label switches the view instantly; no page reload is required. The `Logs` tab remains fully active in the background (polling continues) while the `Circuit` tab is open, so no agent output is ever lost.

The **Circuit tab** renders a track on an canva and animates each car along it in proportional time. Key features include:

- **Start lights sequence**: five red lights illuminate one per second before going out at race start, matching the semaphore agent's timing.
- **Car tokens**: each car is drawn as a coloured dot with its team border glow, a lap-count badge, and a driver label. Position on track reflects the real fraction of the lap elapsed.
- **Pit-stop indicator**: a car entering the pit lane is shown as stationary in the pit area for the duration of the stop.
- **Live sidebar**: an always-visible leaderboard lists cars in current race order with their accumulated times, updated after every lap.
- **Final podium**: once the race ends and all animations have played out, a modal overlay shows the final standings with positions and total times.

Figure 9: Web dashboard — Circuit tab: animated Catmull-Rom track with car positions, start lights and live leaderboard sidebar.

10 Docker Deployment

10.1 Architecture

The Docker Compose setup runs two containers that share a tmux socket volume, allowing the dashboard to read agent panes with zero modifications to the agent logic.

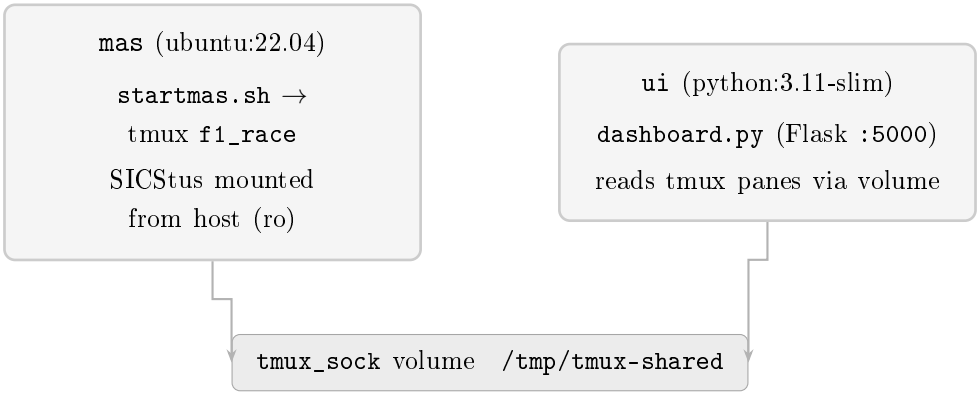

Figure 10: Docker Compose architecture: the `mas` and `ui` containers share a tmux socket volume.

Startup order (managed by Docker healthchecks):

1. `ui` starts immediately and is healthy once `/api/config` responds.
2. `mas` waits for `ui` to be healthy, then runs `startmas.sh`.

10.2 Quickstart

As in the native approach, the configuration requires:

1. Clone the DALI repo from `https://github.com/AAAI-DISIM-UnivAQ/DALI`, checkout the commit to an older version just to be sure that everything works correctly as in mine setup, then navigate to the `DALI/Examples` folder.

   ```
   git checkout 255abcac25db7fe43fdbf7945656240bd61d25f2
   ```

2. Clone the GitHub repo project from `https://github.com/Mik1810/f1_race` under `Examples` folder.

Then, execute the following commands:

```
# 1. Navigate to the project folder
cd DALI/Examples/f1_race

# 2. Auto-detect SICStus and write .env  (run once)
bash docker/setup.sh

# 3. Build and start
docker compose up --build -d
```

The containers need a few seconds to start up and run the agents. Once ready, open `http://localhost:5000` in a browser to view the dashboard.

10.3 Environment Variables

The `mas` container needs the SICSTUS_PATH environment variable to locate the SICStus installation. This is set in `docker-compose.yml` and can be configured via a `.env` file in the project root. This setup is done by the `docker/setup.sh` script, which auto-detects the SICStus path on the host and writes it to `.env`. In case of a different setup of the SICStus installation, the variable can be manually set in `.env` as follows:

Variable	Default	Description
`SICSTUS_PATH`	`/usr/local/sicstus4.6.0`	Host path to SICStus install, mounted read-only into the `mas` container

11 DALI Source Patch System

11.1 Motivation

The F1 Race example depends on three core DALI source files located in `DALI/src/`:

- `active_server_wi.pl`: starts the LINDA Linda server
- `active_user_wi.pl`: starts the interactive user agent
- `active_dali_wi.pl`: the main agent runner (~2200 lines)

These files belong to the upstream DALI framework and are shared across all examples. Modifying them directly in the repository would couple the framework to the F1 example, making future upstream merges error-prone. At the same time, the F1 example requires a number of fixes that are not yet part of the upstream code[3]:

1. **Dynamic port selection**: the server must try ports from 3010 to 3019 and write the bound address to `server.txt` so that every agent connects to the correct port, avoiding `SPIO_E_NET_ADDRINUSE` on restart.
2. **Address propagation to agents**: `active_dali_wi.pl` was hardcoded to `'localhost':3010`; it must now read `server.txt` at startup (`SPIO_E_NET_CONNREFUSED` fix).
3. **User agent entry point**: `active_user_wi.pl` must read `server.txt` and connect only when explicitly invoked via `--goal utente.`, with no top-level directive that could block the process on load.

The *patch system* solves this tension: the upstream files are patched *automatically at runtime*, every time the MAS is started, with no permanent changes stored in the repository.

11.2 Structure

```
f1_race/
  patch/
    apply_dali_wi.sh        <- main patch script
    active_server_wi.pl     <- patched copy of the server file
    active_user_wi.pl       <- patched copy of the user-agent file
```

`active_dali_wi.pl` is *not* copied into `patch/` because it is $\approx$ 2200 lines long; instead a short Python snippet performs a surgical in-place edit.

11.3 Idempotency: the `[F1_PATCH]` Marker

Every patched file contains the following comment on line 7:

```
% [F1_PATCH] read server address from server.txt (dynamic port
    support)
```

Before applying any change, `apply_dali_wi.sh` checks for this marker with `grep`. If it is found, the file is skipped entirely; if not, the patch is applied and the marker is inserted. This makes every invocation of the script safe to repeat without double-patching.

11.4 Reverting the Patches

To revert all three files to their original upstream state, remove the `[F1_PATCH]` marker line and restore the original code. The fastest way is via `git`:

```
git checkout -- src/active_server_wi.pl \
                src/active_user_wi.pl   \
                src/active_dali_wi.pl
```

The next run of `startmas.sh` will re-apply the patches automatically.

[3] I am thinkink of doing a pull request to improve the source code

12 Shutdown

```
# Stop the MAS
tmux kill-session -t f1_race

# Stop the Flask dashboard
# Ctrl+C in the dashboard terminal

# Stop Docker
docker compose down
```